Université Sultan Moulay Slimane
École Nationale des Sciences Appliquées de Khouribga

Filière : Informatique et Ingénierie des Données

RAPPORT DE STAGE

GTM — Galaxy Training Manager

Conception et développement d'une application web de gestion et de pilotage des formations professionnelles

Réalisé par :
— Nougbolo Godwin Elie

Encadré par :
Prince Baba Daouda Coulibaly *(Galaxy Solutions)*
Pr. Nidal Lamghari *(ENSA Khouribga)*

Stage effectué au sein de :

Année universitaire : 2025–2026

Remerciements

Ce stage s'inscrit dans ma formation en Informatique et Ingénierie des Données (IID) à l'ENSA Khouribga.

Je remercie M. Prince Baba Daouda Coulibaly, mon encadrant chez Galaxy Solutions, pour m'avoir confié une mission concrète et pour sa disponibilité. Ses explications sur l'activité de formation m'ont aidé à préciser les règles métier de l'application.

Je remercie également Pr. Nidal Lamghari, mon encadrante à l'ENSA Khouribga, pour son suivi pédagogique de ce stage.

Je remercie Galaxy Solutions pour ce contexte professionnel concret, ainsi que l'équipe pédagogique de la filière Informatique et Ingénierie des Données et mes proches pour leur soutien.

Résumé

Gérer les activités de formation professionnelle — catalogue, sessions, inscriptions, clients et formateurs — devient difficile lorsque les informations sont dispersées dans des fichiers tableurs. Ce rapport présente le travail réalisé chez Galaxy Solutions autour de **GTM (Galaxy Training Manager)**, une application web destinée à centraliser la gestion des formations et à faciliter le pilotage de l'activité.

La solution couvre trois domaines de formation (Web & Data, Management Agile et Cybersécurité). L'application s'appuie sur Flask (Python) et SQLAlchemy pour l'accès à une base MySQL, avec une interface réactive développée en Jinja2, Tailwind CSS et Alpine.js. Trois rôles utilisateurs (Admin, Gestionnaire, Formateur) sont gérés avec des droits strictement contrôlés côté serveur, complétés par un parcours d'onboarding sécurisé sans mot de passe temporaire en clair.

L'application s'articule autour de trois fonctions clés : *Gestion* (suivi opérationnel des entités métier et opérations groupées), *Analyse* (tableau de bord dynamique, graphiques et filtres combinés) et *Décision* (alertes automatiques et analyse en composantes principales des profils clients). Elle intègre des exports en CSV, Excel et PDF, et est validée par une suite de **166 tests automatisés** (100 % de réussite).

Une démarche de relecture critique menée en fin de stage a permis d'auditer la sécurité des accès, d'aligner les seuils d'alerte et de fiabiliser l'application.

Mots-clés : gestion de formation, Flask, SQLAlchemy, RBAC, tableau de bord, KPI, ACP, sécurité applicative, tests automatisés.

Table des matières

Table des figures vii

Liste des tableaux viii

Introduction 1

1 Présentation de l'entreprise et déroulement du stage 2
- 1.1 Galaxy Solutions . . . 2
- 1.2 Déroulement du stage . . . 2
- 1.3 Mon rôle pendant le stage . . . 3
- 1.4 Organisation par socles successifs . . . 3

2 Contexte, problématique et objectifs 4
- 2.1 Cadre de la mission . . . 4
- 2.2 Un besoin de centralisation . . . 4
- 2.3 Les acteurs concernés . . . 4
- 2.4 Problématique . . . 5
- 2.5 Objectifs de la mission . . . 5
  - 2.5.1 Objectif principal . . . 5
  - 2.5.2 Objectifs spécifiques . . . 5
- 2.6 Difficultés identifiées . . . 6
- 2.7 Planning et étapes du projet . . . 6

3 Analyse des besoins et spécifications 7
- 3.1 Introduction . . . 7
- 3.2 État des solutions existantes . . . 7
- 3.3 Besoins fonctionnels . . . 7
  - 3.3.1 Gestion du catalogue et des domaines . . . 7
  - 3.3.2 Gestion des sessions . . . 7
  - 3.3.3 Gestion des clients et des participants . . . 8
  - 3.3.4 Gestion des formateurs . . . 8
  - 3.3.5 Gestion des inscriptions . . . 8
  - 3.3.6 Tableau de bord et pilotage . . . 8
  - 3.3.7 Points d'attention et alertes . . . 8

3.3.8 Exports et analyse multivariée . . . 8
3.4 Besoins non fonctionnels . . . 8
3.4.1 Sécurité et contrôle d'accès (RBAC) . . . 8
3.4.2 Fiabilité des calculs et intégrité des données . . . 9
3.4.3 Ergonomie et performance . . . 9
3.5 Acteurs et cas d'utilisation . . . 9
3.6 Règles de gestion essentielles . . . 9

4 Conception de la solution 11
4.1 Introduction . . . 11
4.2 Architecture fonctionnelle . . . 11
4.2.1 Une organisation en trois temps . . . 11
4.2.2 Architecture en couches . . . 11
4.3 Modèle de données . . . 12
4.3.1 Vue d'ensemble . . . 12
4.4 Contraintes d'intégrité au niveau de la base . . . 12
4.5 Gestion des rôles et des permissions . . . 12
4.5.1 Principe et deux niveaux de contrôle . . . 12
4.5.2 Contrôle côté serveur et isolation du Formateur . . . 15
4.6 Création de compte et activation sécurisée . . . 15
4.7 Règles métier principales . . . 15
4.7.1 Statuts d'une session et taux de remplissage . . . 15
4.7.2 Capacité et liste d'attente . . . 18
4.8 Ce qui est garanti, et par quelle couche . . . 18
4.9 Synthèse de la conception . . . 18

5 Choix technologiques et architecture technique 20
5.1 Introduction . . . 20
5.2 Vue d'ensemble de la pile technique . . . 20
5.3 Python et Flask . . . 20
5.4 SQLAlchemy et MySQL . . . 21
5.5 Flask-Login et Flask-Migrate . . . 21
5.6 Jinja2, Alpine.js et Tailwind CSS . . . 21
5.7 Chart.js, NumPy et pandas . . . 21
5.8 Werkzeug, openpyxl et ReportLab . . . 22
5.9 Sécurité applicative . . . 22
5.10 Trajet d'une requête . . . 22
5.11 Synthèse des choix technologiques . . . 22

6 Implémentation et fonctionnalités 25
6.1 Introduction . . . 25
6.2 Organisation du code . . . 25
6.3 API REST . . . 25
6.4 Exports de données . . . 27
6.5 Tableau de bord . . . 27
6.5.1 Indicateurs clés . . . 27
6.5.2 Filtres et graphiques . . . 27
6.5.3 Espace personnalisé Formateur . . . 27
6.6 Points d'attention . . . 27

6.7 Profil utilisateur 28
6.8 Tests automatisés 28
6.9 Synthèse de l'implémentation 28

7 Analyse décisionnelle et ACP **30**
7.1 Introduction 30
7.2 Principe de l'ACP 30
7.3 Données et méthode de calcul 31
7.4 Résultats obtenus sur le jeu de données 31
7.5 Qualité de représentation et contributions 32
7.6 Synthèse automatique et seuils de rigueur 32
7.7 Limites de l'analyse 32
7.8 Bilan 34

8 Tests, audit et bilan critique **35**
8.1 Introduction 35
8.2 Démonstration fonctionnelle 35
8.3 Résultats des tests automatisés 35
8.4 Bilan critique et audit interne 37
8.4.1 Démarche de vérification 37
8.4.2 Points forts confirmés 37
8.4.3 Limite n°1 : calcul du statut d'activité client 37
8.4.4 Limite n°2 : harmonisation documentaire 37
8.5 Autres limites et perspectives 37
8.6 Conclusion du bilan 39

Conclusion et perspectives **40**

Table des figures

4.1 Architecture fonctionnelle générale de GTM : Gestion, Analyse, Décision . . 13
4.2 Diagramme de classes du modèle de données de GTM 13
4.3 Modèle logique de données de GTM 14
4.4 Catalogue des formations dans GTM 14
4.5 Fiche détaillée d'une formation avec l'historique de ses sessions 16
4.6 Création d'un compte par l'Admin, sans mot de passe 16
4.7 Email d'invitation reçu par le nouvel utilisateur 17
4.8 Page d'activation : choix du mot de passe définitif 17

5.1 Trajet d'une requête entre les différentes technologies utilisées 23

6.1 Profil de l'utilisateur connecté et accès au changement de mot de passe . . . 29

7.1 Représentation des clients sur le plan factoriel $(F1, F2)$ 33
7.2 Coordonnées, $\cos^2$ et contributions calculés pour les clients 33

8.1 Tableau de bord — vue Admin/Gestionnaire 36
8.2 Fiche détaillée d'une session avec taux de remplissage 36
8.3 Vue d'un compte Formateur, restreinte à son propre périmètre 38

Liste des tableaux

3.1 Comparaison de solutions existantes avec les besoins de GTM 10
3.2 Matrice des cas d'utilisation selon les rôles 10

4.1 Niveau réel de garantie de quelques règles importantes de GTM 19

5.1 Technologies utilisées dans GTM 24

6.1 Organisation des principaux dossiers de GTM 26
6.2 Codes HTTP utilisés par l'API GTM et leur signification 26

8.1 Alignement des règles de points d'attention 38

Introduction

Une entreprise de formation professionnelle gère chaque jour un grand nombre d'informations liées entre elles : les formations proposées, les sessions planifiées, les clients qui inscrivent leurs salariés, les participants, les formateurs et les inscriptions. Quand ces informations sont réparties dans plusieurs fichiers séparés, il devient difficile de répondre rapidement à des questions de gestion : une session a-t-elle encore de la place ? un client est-il toujours actif ? quel domaine est le plus demandé ?

Ce stage a porté sur une réponse concrète à ce besoin à travers **GTM (Galaxy Training Manager)**, une application web développée pour Galaxy Solutions. L'objectif était de centraliser l'activité de formation, de faciliter son suivi quotidien et d'apporter des indicateurs fiables pour l'aide à la décision.

Le stage s'est déroulé à distance. À partir du cahier des charges fourni par l'entreprise, mon travail a consisté à analyser les besoins, à concevoir l'architecture, puis à développer et tester l'ensemble des fonctionnalités.

Le rapport est organisé en huit chapitres : le **chapitre 1** présente Galaxy Solutions et le déroulement du stage ; le **chapitre 2** expose le contexte, la problématique et les objectifs ; le **chapitre 3** détaille les besoins fonctionnels et non fonctionnels ; le **chapitre 4** présente la conception et le modèle de données ; le **chapitre 5** justifie les choix technologiques ; le **chapitre 6** décrit l'implémentation et les fonctionnalités réalisées ; le **chapitre 7** présente l'analyse en composantes principales ; enfin, le **chapitre 8** détaille la suite de tests, l'audit critique et les perspectives.

Chapitre 1

Présentation de l'entreprise et déroulement du stage

1.1 Galaxy Solutions

Galaxy Solutions est un cabinet marocain de conseil et de formation professionnelle, basé à Mohammedia. L'entreprise travaille en B2B (*business to business*) : elle propose ses services à d'autres entreprises clientes pour la montée en compétences de leurs collaborateurs.

Son catalogue de formation couvre plusieurs domaines, notamment la QHSE (qualité, hygiène, sécurité et environnement), le management, le digital et l'informatique. Les formations sont organisées selon deux modalités :

- **Intra-entreprise :** une session dédiée aux salariés d'une seule entreprise cliente.
- **Inter-entreprises :** une session commune regroupant des salariés de plusieurs entreprises clientes distinctes.

Dans le cadre de ce stage, le périmètre retenu pour l'application **GTM (Galaxy Training Manager)** porte sur trois domaines représentatifs : **Web & Data**, **Management Agile** et **Cybersécurité**. Ce périmètre permettait de couvrir l'ensemble des cas de gestion tout en restant adapté au temps imparti.

1.2 Déroulement du stage

Le stage s'est déroulé à distance. La convention de stage fixait un début administratif au 1er juillet 2026, et le cadrage technique définitif m'a été transmis le 26 juillet 2026.

Dès la réception du cahier des charges, j'ai analysé les exigences, les priorités et les contraintes techniques afin de structurer le travail. Les échanges avec mon encadrant chez Galaxy Solutions avaient lieu régulièrement par WhatsApp, pour valider les orientations et clarifier les règles métier au besoin.

Le développement devait être réalisé sur environ quatre semaines. Cette contrainte m'a conduit à organiser le projet en étapes progressives : bâtir d'abord une base solide, la tester

et la fiabiliser, avant d'ajouter les fonctionnalités de suivi et d'analyse.

1.3 Mon rôle pendant le stage

J'ai assuré l'ensemble du cycle de conception et de réalisation de GTM en autonomie, sous la supervision de mon encadrant.

Mes activités ont couvert les étapes suivantes :

— analyse et formalisation des besoins fonctionnels ;
— modélisation des données relationnelles et conception de l'architecture ;
— définition des rôles utilisateurs et des règles de sécurité ;
— développement de l'API REST et de l'interface web ;
— implémentation des calculs statistiques, du tableau de bord et du module ACP ;
— réalisation de la suite de 166 tests automatisés ;
— relecture critique du code et fiabilisation de l'application.

1.4 Organisation par socles successifs

La principale difficulté de la mission résidait dans la gestion du temps face à des composants fortement interdépendants (base de données, sécurité, logique métier, interface, indicateurs).

Pour garantir la qualité du livrable, j'ai adopté une démarche basée sur des **socles successifs** :

1. **Socle 1 (Fondation) :** modélisation de la base, gestion des entités et sécurisation des accès.
2. **Socle 2 (Métier & Pilotage) :** règles de gestion, calcul des indicateurs et tableau de bord.
3. **Socle 3 (Analytique & Consolidation) :** analyse ACP, exports documentaires et validation globale.

Cette démarche a permis de sécuriser le développement à chaque étape et d'éviter l'accumulation de bugs en fin de projet. Elle illustre un principe essentiel de l'ingénierie logicielle : la solidité du socle métier et l'exactitude des calculs priment sur la multiplication de fonctionnalités non stabilisées.

Chapitre 2

Contexte, problématique et objectifs

2.1 Cadre de la mission

Le périmètre confié pour l'application **GTM** porte sur trois domaines pédagogiques clés de Galaxy Solutions : **Web & Data**, **Management Agile** et **Cybersécurité**.

Les formations sont dispensées par des formateurs internes ou externes auprès de salariés d'entreprises clientes. L'outil doit permettre de centraliser l'ensemble de ces informations et de suivre l'activité en temps réel.

2.2 Un besoin de centralisation

Avant la mise en place de GTM, les données relatives aux formations, sessions, inscriptions et clients étaient dispersées dans plusieurs fichiers tableurs gérés indépendamment.

Cette dispersion entraînait plusieurs difficultés opérationnelles :

— absence de vision consolidée sur l'activité globale ;
— lenteur pour préparer les bilans clients (historique, salariés formés, dernière activité) ;
— difficulté à connaître immédiatement le taux de remplissage d'une session ou le nombre de places restantes ;
— risque d'incohérence entre les différents fichiers de suivi.

L'objectif de GTM est de remplacer ces fichiers séparés par une application web adossée à une base de données relationnelle unique.

2.3 Les acteurs concernés

L'application distingue trois profils d'utilisateurs :

— **L'Admin :** responsable technique, il dispose de tous les droits, notamment la création et la gestion des comptes utilisateurs.
— **Le Gestionnaire :** responsable opérationnel, il gère le catalogue, les sessions, les clients, les participants, les inscriptions et dispose de l'accès complet au tableau de bord, aux alertes et aux exports.
— **Le Formateur :** intervenant pédagogique, il accède exclusivement en consultation aux sessions qu'il anime et bénéficie d'un tableau de bord personnel.

2.4 Problématique

La problématique du projet peut se formuler ainsi :

> *Comment concevoir et développer une application web capable de centraliser la gestion des formations, des sessions, des clients et des inscriptions, tout en garantissant un contrôle d'accès strict par rôle et en fournissant des indicateurs fiables d'aide à la décision ?*

Cette problématique s'articule autour de trois enjeux majeurs : la **modélisation** des données relationnelles, la **sécurité** des accès côté serveur et la **fiabilité** des indicateurs de pilotage.

2.5 Objectifs de la mission

2.5.1 Objectif principal

Concevoir et développer une application web complète, robuste et sécurisée pour centraliser la gestion et le pilotage des formations de Galaxy Solutions.

2.5.2 Objectifs spécifiques

1. **Modéliser** une base relationnelle normalisée sous MySQL.
2. **Développer** une API REST avec Flask et SQLAlchemy pour toutes les opérations de gestion.
3. **Mettre en place** un contrôle d'accès par rôle (RBAC) vérifié au niveau backend.
4. **Gérer** les contraintes métier (capacités de session, listes d'attente, intégrité référentielle lors des suppressions).
5. **Construire** un tableau de bord avec indicateurs clés (KPI), graphiques et filtres combinés.
6. **Intégrer** un module d'Analyse en Composantes Principales (ACP) pour explorer les profils clients.
7. **Générer** des exports professionnels en CSV, Excel et PDF.
8. **Valider** l'application à l'aide d'une suite de tests automatisés complète.

2.6 Difficultés identifiées

La première difficulté concernait la **gestion du temps** sur un projet intégrant à la fois du développement web, de la sécurité et de l'analyse de données.

La deuxième portait sur la **rigueur des permissions** : l'isolation du Formateur doit être absolue et s'appliquer sur chaque route et chaque traitement de données.

La troisième portait sur l'**exactitude des indicateurs** : définir clairement les règles de calcul (par exemple, calculer le remplissage uniquement sur les inscriptions confirmées) pour garantir des chiffres vérifiables et cohérents sur toutes les pages.

2.7 Planning et étapes du projet

En accord avec les priorités du cahier des charges, le travail s'est articulé en quatre semaines :

- **Semaine 1 (Conception) :** modèle relationnel, diagramme de classes et script de données de démonstration.
- **Semaine 2 (Backend & Sécurité) :** modèles SQLAlchemy, authentification, onboarding par token et endpoints d'API.
- **Semaine 3 (Pilotage & Restitution) :** services de calcul, filtres, tableau de bord et alertes.
- **Semaine 4 (Analytique & Audit) :** module ACP, exports, suite de tests automatisés et relecture critique.

Chapitre 3

Analyse des besoins et spécifications

3.1 Introduction

Ce chapitre présente l'analyse détaillée des besoins auxquels l'application GTM doit répondre, en distinguant les fonctionnalités attendues (besoins fonctionnels) et les contraintes techniques et de qualité (besoins non fonctionnels).

3.2 État des solutions existantes

Avant d'engager un développement sur mesure, trois solutions du marché ont été étudiées (tableau 3.1).

Ce comparatif confirme l'intérêt d'une application sur mesure centrée sur les relations entre formations, sessions, entreprises clientes, salariés et formateurs.

3.3 Besoins fonctionnels

3.3.1 Gestion du catalogue et des domaines

L'application permet de gérer les formations (titre, durée en jours, description, domaine d'expertise). Une formation ne peut être supprimée que si aucune session n'y est rattachée.

Les trois domaines (Web & Data, Management Agile, Cybersécurité) disposent de leur propre gestion et servent de critère de regroupement pour les filtres et les graphiques.

3.3.2 Gestion des sessions

Une session représente la planification d'une formation avec ses dates, son lieu, son type (intra ou inter-entreprises), son formateur et sa capacité maximale.

Son statut évolue entre `planifiee`, `en_cours`, `terminee` (selon les dates) ou `annulee` (décision explicite).

3.3.3 Gestion des clients et des participants

Un **client** représente une entreprise cliente. L'application permet de suivre ses salariés inscrits, ses formations suivies et son niveau d'activité récente.

Un **participant** représente un salarié rattaché à une entreprise. En plus de la création individuelle, l'application propose un **import par lot** depuis un fichier CSV ou XLSX avec contrôle des données (emails valides, détection des doublons).

3.3.4 Gestion des formateurs

Les formateurs (internes ou externes) sont répertoriés avec leur domaine de compétence. Pour un formateur interne, le compte est rattaché à un utilisateur du système.

3.3.5 Gestion des inscriptions

Une inscription associe un participant à une session avec un statut : `confirmee`, `liste_attente` ou `annulee`. Le système garantit qu'un participant ne peut pas être inscrit deux fois à la même session.

3.3.6 Tableau de bord et pilotage

L'application offre un tableau de bord synthétique avec 6 indicateurs clés, des graphiques d'activité et des filtres combinés (année, domaine, client, formateur).

Le rôle Formateur dispose d'une vue personnalisée (`/dashboard-formateur`) limitée à ses propres chiffres d'activité.

3.3.7 Points d'attention et alertes

Le système détecte automatiquement les situations nécessitant une action :

- **Session à risque (danger) :** session débutant dans les 7 jours avec un remplissage inférieur à 40 %.
- **Client inactif (warning) :** client sans inscription confirmée depuis au moins 6 mois.
- **Tendance d'activité (info) :** variation du volume d'inscriptions supérieure à 5 % sur les 3 derniers mois.

3.3.8 Exports et analyse multivariée

L'ensemble des modules propose des exports en CSV, Excel (XLSX) stylisé et PDF (feuille d'émargement et rapport de synthèse).

L'Analyse en Composantes Principales (ACP) permet d'analyser les ressemblances entre entreprises clientes en fonction des formations suivies.

3.4 Besoins non fonctionnels

3.4.1 Sécurité et contrôle d'accès (RBAC)

Chaque accès est soumis à authentification. Les permissions sont strictement vérifiées côté serveur sur chaque route d'API. Le Formateur est isolé dans son périmètre de consultation, et toute tentative d'écriture lui est interdite (code HTTP 403).

3.4.2 Fiabilité des calculs et intégrité des données

Les calculs d'indicateurs sont centralisés côté backend pour garantir des résultats rigoureusement identiques sur toutes les vues. L'intégrité référentielle empêche les suppressions orphelines.

3.4.3 Ergonomie et performance

L'interface propose une navigation claire, responsive, avec des retours visuels lors des actions (états de chargement, notifications Toast).

3.5 Acteurs et cas d'utilisation

Le tableau 3.2 synthétise les droits par rôle.

3.6 Règles de gestion essentielles

1. Unicité stricte de l'inscription d'un participant à une session.
2. Seules les inscriptions confirmées sont comptabilisées dans la capacité.
3. Rejet (HTTP 409) de toute inscription confirmée si la session est complète.
4. Inscription en liste d'attente possible sans impacter la capacité.
5. Suppression protégée : impossible de supprimer une entité ayant des dépendances actives.
6. Un client est qualifié d'inactif s'il n'a aucune inscription confirmée depuis six mois.

Solution	Points forts	Limites par rapport au besoin étudié
Moodle	Plateforme largement utilisée pour l'apprentissage en ligne, open source [1].	Orientée gestion de contenus et cours en ligne. Moins adaptée au suivi direct des entreprises clientes et de la logistique des sessions en présentiel.
TalentLMS	Gestion conviviale des cours et des apprenants [2].	Centrée sur le e-learning individuel plutôt que sur les relations B2B, les formateurs et les capacités de session.
Docebo	Plateforme d'entreprise très complète [3].	Complexité et coût élevés pour une PME. Un outil dédié permet un contrôle total sur le modèle métier et les calculs analytiques.

TABLE 3.1 – Comparaison de solutions existantes avec les besoins de GTM

Fonctionnalité	Admin	Gestionnaire	Formateur
Gestion des comptes utilisateurs	✓		
Gestion des formations et domaines	✓	✓	
Gestion des sessions	✓	✓	
Consultation des sessions	✓	✓	✓(ses sessions)
Gestion des clients et participants	✓	✓	
Import par lot de participants	✓	✓	
Gestion des inscriptions	✓	✓	
Tableau de bord global	✓	✓	
Tableau de bord personnel			✓
Module ACP	✓	✓	
Exports documentaires (CSV, XLSX, PDF)	✓	✓	✓(restreint)

TABLE 3.2 – Matrice des cas d'utilisation selon les rôles

Chapitre 4

Conception de la solution

4.1 Introduction

Ce chapitre explique comment les besoins présentés au chapitre 3 ont été traduits en une conception technique cohérente. Elle porte sur quatre éléments : l'organisation générale de la solution, le modèle de données, le système de rôles et de permissions, et les règles métier importantes. Les technologies utilisées sont détaillées au chapitre 5.

4.2 Architecture fonctionnelle

4.2.1 Une organisation en trois temps

GTM est organisé autour de trois fonctions complémentaires (figure 4.1) :

— **Gestion** : les opérations quotidiennes — création d'une formation, planification d'une session, enregistrement d'un client, inscription d'un participant.

— **Analyse** : les données enregistrées sont transformées en indicateurs, graphiques et résultats filtrés.

— **Décision** : les informations produites servent à identifier des situations nécessitant une action, comme un client inactif ou une session insuffisamment remplie. L'ACP apporte une représentation exploratoire des profils de clients.

4.2.2 Architecture en couches

L'organisation interne du code sépare les **modèles** (entités et relations de la base de données), les **services** (logique métier réutilisable : règles de visibilité, validations, calculs statistiques) et les **routes** (points d'entrée qui reçoivent les requêtes, effectuent les contrôles, appellent les services et construisent les réponses).

Cette séparation limite le mélange des responsabilités. La détermination des données visibles par un Formateur, par exemple, est centralisée dans un service commun plutôt que

reproduite dans chaque route, ce qui réduit les duplications de code et facilite la maintenance.

4.3 Modèle de données

4.3.1 Vue d'ensemble

Le modèle de données de GTM comprend neuf entités principales : `Role`, `Domaine`, `Utilisateur`, `Formateur`, `Formation`, `Session`, `Client`, `Participant` et `Inscription`.

`Domaine` est une table de référence utilisée pour structurer les trois domaines de formations. Elle évite d'utiliser un champ texte libre et limite les erreurs de saisie.

`Formation` représente un élément du catalogue (intitulé, durée, description, domaine). `Session` représente une réalisation précise de cette formation, avec ses propres dates, son lieu, son formateur, sa capacité et son statut. Une même formation peut donc être utilisée pour plusieurs sessions à des dates différentes (figures 4.4 et 4.5).

`Formateur` représente une personne chargée d'animer une ou plusieurs sessions. Le lien avec un compte `Utilisateur` est optionnel, ce qui permet de représenter un formateur externe sans accès à l'application.

`Client` représente l'entreprise qui fait appel à Galaxy Solutions. `Participant` représente une personne travaillant pour ce client et participant à une formation. Un client peut avoir plusieurs participants et être associé à plusieurs sessions et inscriptions. Les participants ne sont pas des utilisateurs de GTM.

`Inscription` relie un participant à une session, avec un statut : `confirmee`, `annulee` ou `liste_attente`. Cette distinction est essentielle pour les règles métier et le calcul du taux de remplissage.

`Utilisateur` représente un compte pouvant se connecter à l'application. Le mot de passe n'est jamais stocké en clair, seule une version hachée est conservée. La création et l'activation des comptes sont présentées en section 4.6.

4.4 Contraintes d'intégrité au niveau de la base

Une contrainte d'unicité sur le couple (`session_id`, `participant_id`) empêche qu'un même participant soit inscrit deux fois à la même session. Une autre contrainte d'unicité sur `utilisateur_id` dans `Formateur` empêche qu'un même compte soit associé à plusieurs formateurs.

Ces règles imposées au niveau de la base ajoutent une protection indépendante de l'interface et des routes : une requête incorrecte ne peut pas simplement contourner une règle protégée en base. Une contrainte protège cependant uniquement le cas explicitement défini ; les autres règles métier doivent être garanties par l'application elle-même.

4.5 Gestion des rôles et des permissions

4.5.1 Principe et deux niveaux de contrôle

GTM utilise un modèle de contrôle d'accès basé sur les rôles, le **RBAC** (*Role-Based Access Control*) : les autorisations sont associées à des rôles, puis les rôles sont attribués aux utilisateurs [**?**, **?**]. Les trois rôles sont Admin, Gestionnaire et Formateur.

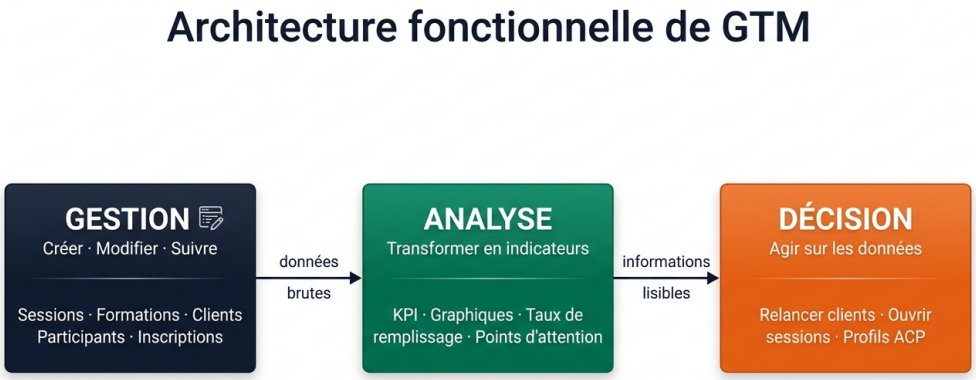


FIGURE 4.1 – Architecture fonctionnelle générale de GTM : Gestion, Analyse, Décision

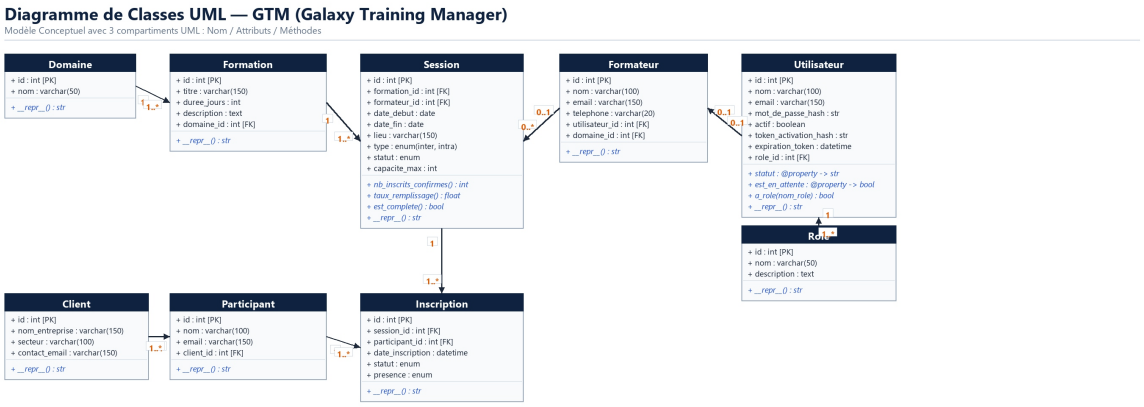


FIGURE 4.2 – Diagramme de classes du modèle de données de GTM

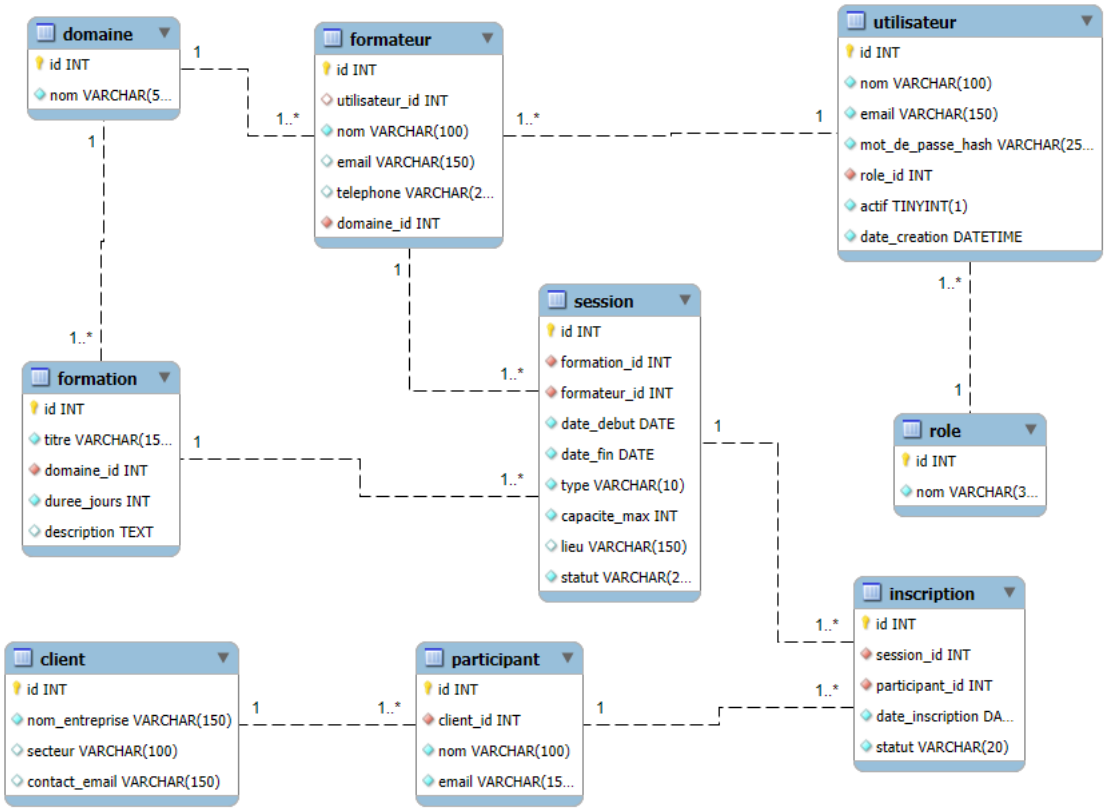


FIGURE 4.3 – Modèle logique de données de GTM

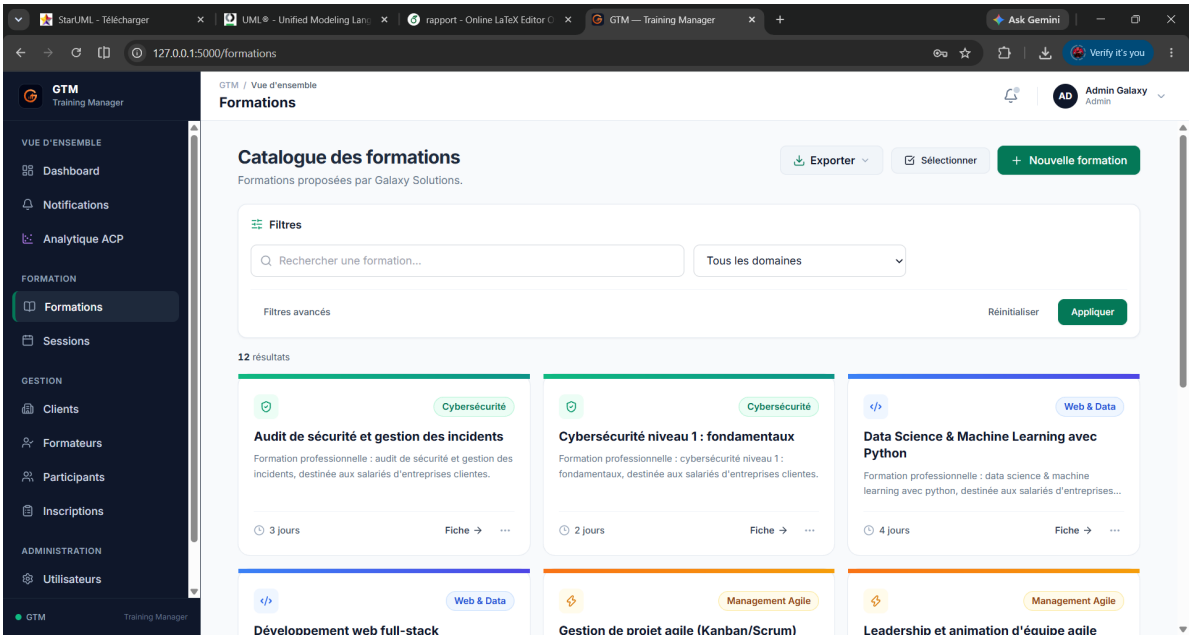


FIGURE 4.4 – Catalogue des formations dans GTM

Deux questions doivent être distinguées : l'utilisateur a-t-il le droit d'effectuer cette action (par exemple, la création d'une session est réservée à l'Admin et au Gestionnaire) et, si oui, quelles données peut-il consulter (la consultation des sessions est autorisée au Formateur, mais uniquement pour ses propres sessions) ? Cette distinction entre *autorisation* et *périmètre des données* est essentielle dans la conception de l'application.

4.5.2 Contrôle côté serveur et isolation du Formateur

Les permissions d'action sont appliquées dans les routes. Le filtrage du périmètre des données est pris en charge par des services de visibilité, pour éviter qu'un Formateur puisse modifier un identifiant dans l'URL et accéder à une ressource hors de ses sessions. Le serveur reste l'autorité finale pour les permissions et les règles métier, conformément aux principes généraux de sécurité des applications web [4, 5].

La règle principale pour un Formateur est :

> *Un Formateur ne doit accéder qu'aux données qui correspondent à ses propres sessions.*

Cette règle concerne les sessions, les participants associés, les inscriptions correspondantes, les clients accessibles à travers ces relations, et les indicateurs qui utilisent ces données. L'ensemble de ces règles d'accès est audité et validé au chapitre 8.

4.6 Création de compte et activation sécurisée

La création d'un compte suit un parcours séparé de l'utilisation normale de l'application, pour éviter la transmission d'un mot de passe temporaire entre l'Admin et le nouvel utilisateur.

1. **Création du compte :** l'Admin crée le compte avec le nom, l'adresse électronique et le rôle, sans mot de passe définitif (figure 4.6).
2. **Génération du token :** le serveur génère un token aléatoire avec le module `secrets` de Python [6]. Seule une empreinte SHA-256 du token est enregistrée, et le token n'est transmis qu'une fois, dans le lien d'activation.
3. **Choix du mot de passe :** le nouvel utilisateur accède au lien d'activation et choisit son mot de passe définitif (figures 4.7 et 4.8), stocké sous forme de hash.

Lors de l'activation, le serveur vérifie que le token présenté correspond au token attendu, qu'il n'a pas déjà été utilisé et qu'il n'est pas expiré. La comparaison utilise `hmac.compare_digest`, une comparaison en temps constant [7]. Le token expire après 48 heures et ne peut plus être réutilisé une fois le compte activé.

4.7 Règles métier principales

4.7.1 Statuts d'une session et taux de remplissage

Une session possède quatre statuts : `planifiee`, `en_cours`, `terminee`, `annulee`. La validation du statut tient compte des dates : une session ne peut pas être déclarée terminée si sa date de fin n'est pas encore passée.

Le taux de remplissage d'une session est défini par :

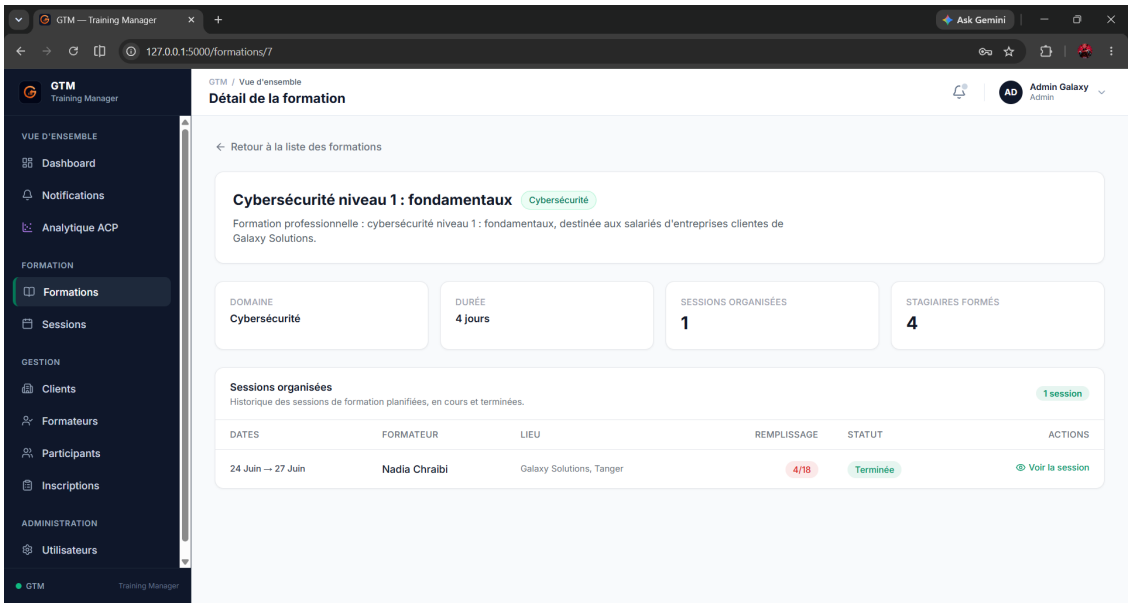


FIGURE 4.5 – Fiche détaillée d'une formation avec l'historique de ses sessions

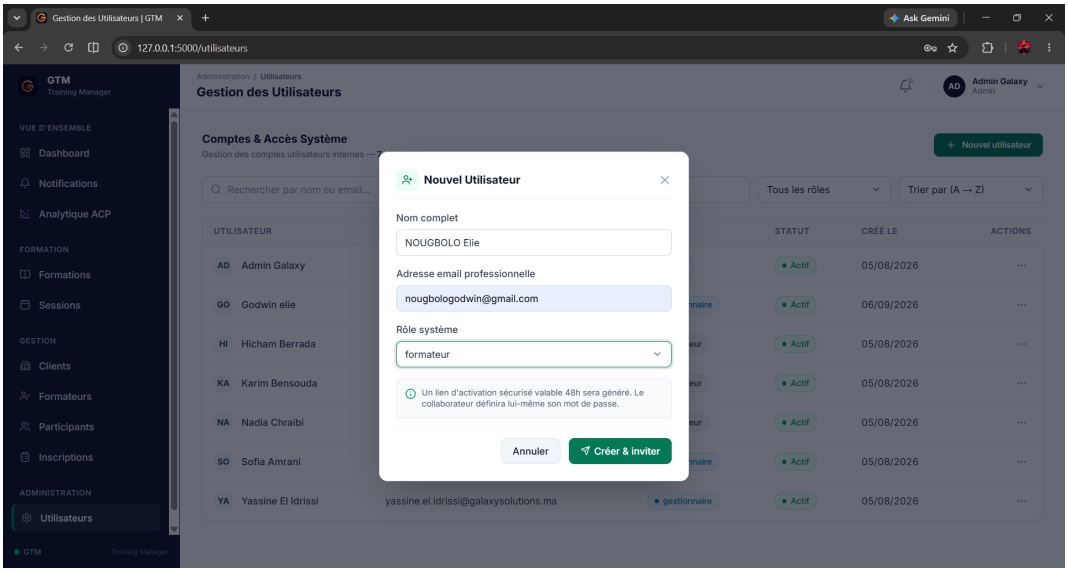


FIGURE 4.6 – Création d'un compte par l'Admin, sans mot de passe

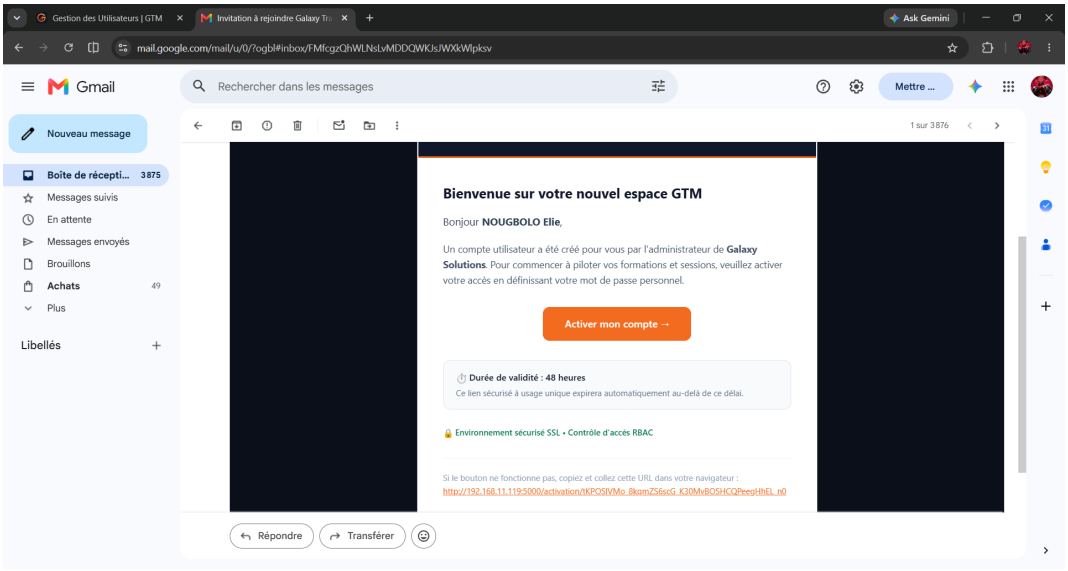


FIGURE 4.7 – Email d'invitation reçu par le nouvel utilisateur

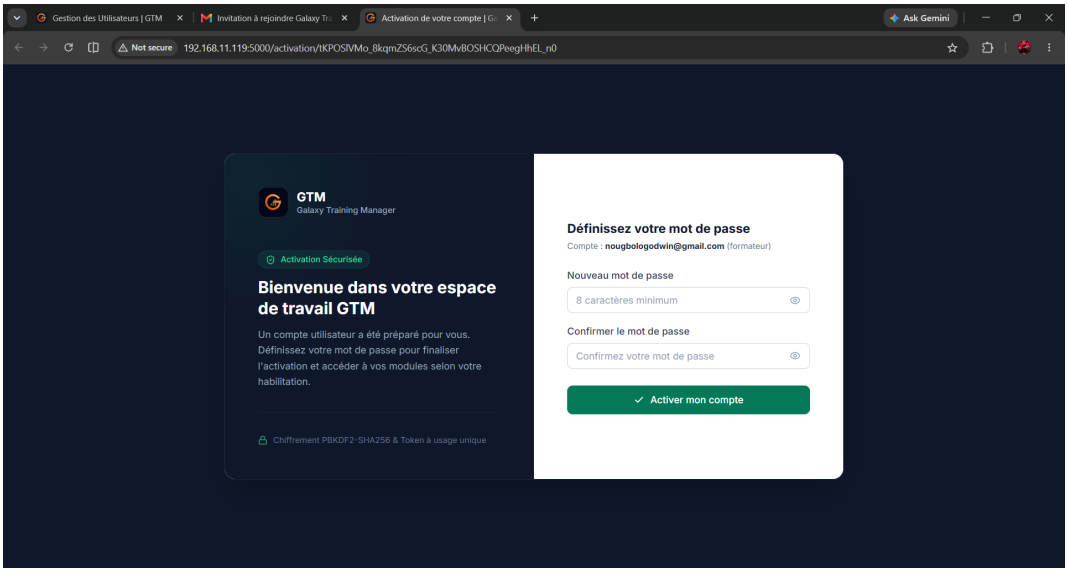


FIGURE 4.8 – Page d'activation : choix du mot de passe définitif

$$\tau_{\text{remplissage}} = \frac{N_{\text{confirmees}}}{C_{\max}} \tag{4.1}$$

où $N_{\text{confirmees}}$ est le nombre d'inscriptions confirmées et $C_{\max}$ la capacité maximale. Seules les inscriptions confirmées sont prises en compte : une inscription annulée ne correspond plus à une place occupée, et une inscription en liste d'attente ne correspond pas encore à une place confirmée.

Dans le jeu de données de démonstration, la session n° 1 (« Développement web full-stack ») a 20 places et 15 inscriptions confirmées, soit un taux de 75 %. La session n° 56 (« SQL avancé et modélisation de données ») a 20 places et 20 inscriptions confirmées, elle est donc complète.

4.7.2 Capacité et liste d'attente

Lorsque le nombre d'inscriptions confirmées atteint la capacité maximale, une nouvelle inscription confirmée n'est pas acceptée : le serveur répond avec un code HTTP 409 (*Conflict*), et le gestionnaire peut inscrire le participant en liste d'attente. Cette règle est contrôlée côté serveur, car une interface qui désactive simplement un bouton ne suffit pas [4, 5].

4.8 Ce qui est garanti, et par quelle couche

Le tableau 4.1 présente le niveau principal auquel certaines règles importantes de GTM sont garanties.

Ce tableau montre une distinction importante entre **la définition d'une règle** et **sa garantie réelle** : centraliser le filtrage des données dans un service réduit le risque de duplication, mais chaque fonction qui exploite des données agrégées doit aussi respecter ce périmètre.

4.9 Synthèse de la conception

La conception de GTM repose sur : un modèle relationnel représentant les relations métier ; une architecture en couches séparant modèles, logique métier et routes ; un contrôle d'accès distinguant les droits d'action du périmètre des données ; des contraintes d'intégrité au niveau de la base pour certaines règles ; une validation côté serveur des principales règles métier ; et un parcours d'activation de compte séparé de la création classique des utilisateurs. Les choix technologiques permettant de réaliser cette architecture sont détaillés au chapitre 5.

Règle	**Niveau de garantie**
Pas de double inscription à une même session	Base de données, par contrainte d'unicité
Capacité maximale d'une session	Service métier appelé lors de l'inscription
Cohérence du statut d'une session avec les dates	Service de validation métier
Token d'activation à usage unique et limité dans le temps	Service métier et stockage de l'état du token
Isolation des données du Formateur	Service de visibilité et contrôles appliqués aux traitements concernés
Droit d'effectuer une action selon le rôle	Contrôles d'autorisation appliqués aux routes

TABLE 4.1 – Niveau réel de garantie de quelques règles importantes de GTM

Chapitre 5

Choix technologiques et architecture technique

5.1 Introduction

Ce chapitre présente les technologies retenues pour réaliser GTM et les raisons de ces choix, en tenant compte du périmètre de la mission, du temps disponible et des fonctionnalités attendues.

5.2 Vue d'ensemble de la pile technique

L'ensemble a été choisi pour construire une application web suffisamment complète tout en gardant une architecture simple à comprendre et à maintenir.

5.3 Python et Flask

Python est le langage principal côté serveur : il donne accès à un écosystème adapté au développement web, à la manipulation de données et aux traitements statistiques, notamment avec pandas et NumPy pour l'ACP.

Flask est utilisé comme framework web pour recevoir les requêtes HTTP, les orienter vers les routes et construire les réponses [8]. Django aurait été une alternative sérieuse, avec plus de fonctionnalités intégrées (administration, formulaires, mécanismes déjà configurés), mais ces fonctionnalités n'étaient pas toutes nécessaires pour le périmètre de GTM. Flask permet de sélectionner les composants réellement utiles et de garder une organisation explicite des routes et des services, mieux adaptée à la taille du projet.

5.4 SQLAlchemy et MySQL

SQLAlchemy est un ORM (*Object-Relational Mapping*) qui représente les tables de la base sous forme de classes Python [9]. L'alternative aurait été d'écrire directement les requêtes SQL dans les différentes parties de l'application, ce qui aurait dispersé davantage la logique d'accès aux données. Avec SQLAlchemy, les modèles représentent les entités et les services peuvent les manipuler sans dépendre de la syntaxe SQL pour chaque opération courante, ce qui facilite la séparation présentée au chapitre 4.

MySQL stocke durablement les données [10]. Les entités de GTM sont fortement liées entre elles (une formation appartient à un domaine, une session à une formation et à un formateur, etc.), ce qui correspond bien à une base relationnelle, et permet d'imposer certaines contraintes d'intégrité directement au niveau du stockage (chapitre 4).

5.5 Flask-Login et Flask-Migrate

Flask-Login gère l'authentification et la session de l'utilisateur connecté [11], et s'intègre avec le système de rôles : une fois authentifié, les contrôles d'autorisation déterminent si le rôle de l'utilisateur permet l'action demandée.

Flask-Migrate, basé sur Alembic, gère les migrations de schéma [12, 13], ce qui évite de modifier manuellement la structure de la base à chaque évolution du modèle et facilite la synchronisation entre le code et le schéma de données.

5.6 Jinja2, Alpine.js et Tailwind CSS

Jinja2 génère les pages HTML à partir de gabarits réutilisables [14], ce qui permet de partager la navigation, les en-têtes et les éléments communs aux différentes pages. Alpine.js apporte de petites interactions côté navigateur sans framework complet [15], notamment pour les filtres dynamiques et les fenêtres modales.

Une SPA (*Single Page Application*) avec React ou Vue aurait été une autre possibilité, mais la majorité des écrans de GTM repose sur des listes, des formulaires et des fiches détaillées, avec des besoins d'interactivité ciblés. Jinja2 avec Alpine.js permet donc de garder une architecture plus simple tout en offrant l'interactivité nécessaire, ce qui correspond au principe de sobriété énoncé dans le cahier des charges.

Tailwind CSS construit l'interface à partir de classes utilitaires [16], ce qui permet de développer rapidement une interface cohérente pour les formulaires, tableaux, cartes d'indicateurs et éléments de navigation.

5.7 Chart.js, NumPy et pandas

Chart.js représente graphiquement les résultats du tableau de bord [17] : les données statistiques sont calculées côté serveur, puis transmises à l'interface pour être affichées sous forme de graphiques, sans introduire un framework front-end complet uniquement pour cela.

NumPy et pandas préparent les données et réalisent les calculs de l'ACP [18, **?**] : pandas facilite la construction de la matrice clients × formations, NumPy fournit les opérations numériques pour la standardisation et les calculs matriciels. Le détail du calcul est présenté au chapitre 7, à partir des principes décrits par Jolliffe et Cadima [19].

5.8 Werkzeug, openpyxl et ReportLab

Werkzeug fournit les mécanismes de hachage des mots de passe utilisés par Flask [?] : le mot de passe n'est jamais conservé en clair. Utiliser une bibliothèque spécialisée évite les erreurs qu'une implémentation manuelle pourrait introduire (sel oublié, paramètres mal choisis).

openpyxl crée les fichiers Excel au format XLSX [?], avec une mise en forme adaptée à la lecture (en-têtes, largeurs de colonnes). ReportLab génère les documents PDF [?], notamment une feuille d'émargement par session et un rapport de synthèse à partir du tableau de bord. Le CSV reste utile pour les données brutes et leur traitement dans un tableur.

5.9 Sécurité applicative

Les mécanismes de sécurité de GTM s'inspirent des bonnes pratiques générales de l'OWASP [4, 5] : authentification des utilisateurs, contrôle des accès par rôle, restriction du périmètre des données du Formateur, hachage des mots de passe, parcours d'activation par token, validation des données côté serveur. Les perspectives de renforcement de cette sécurité sont analysées au chapitre 8.

5.10 Trajet d'une requête

Lorsqu'un utilisateur effectue une action, le navigateur envoie une requête HTTP à Flask. La route reçoit la requête et effectue les premiers contrôles (authentification, permissions), puis appelle le service métier approprié, qui applique les règles nécessaires et utilise SQLAlchemy pour lire ou modifier les données dans MySQL. Le résultat est renvoyé en HTML ou en JSON ; pour le tableau de bord, les données statistiques renvoyées sont ensuite utilisées par Chart.js pour produire les graphiques (figure 5.1).

5.11 Synthèse des choix technologiques

Les choix retenus répondent à une logique commune : utiliser des composants suffisamment spécialisés pour répondre aux besoins de la mission, tout en gardant une architecture relativement simple. Flask assure la couche web, SQLAlchemy facilite l'accès à MySQL, Jinja2 et Alpine.js construisent une interface interactive sans SPA complète, et NumPy avec pandas apportent les outils nécessaires à l'analyse des données. Cette organisation correspond à l'architecture présentée au chapitre 4.

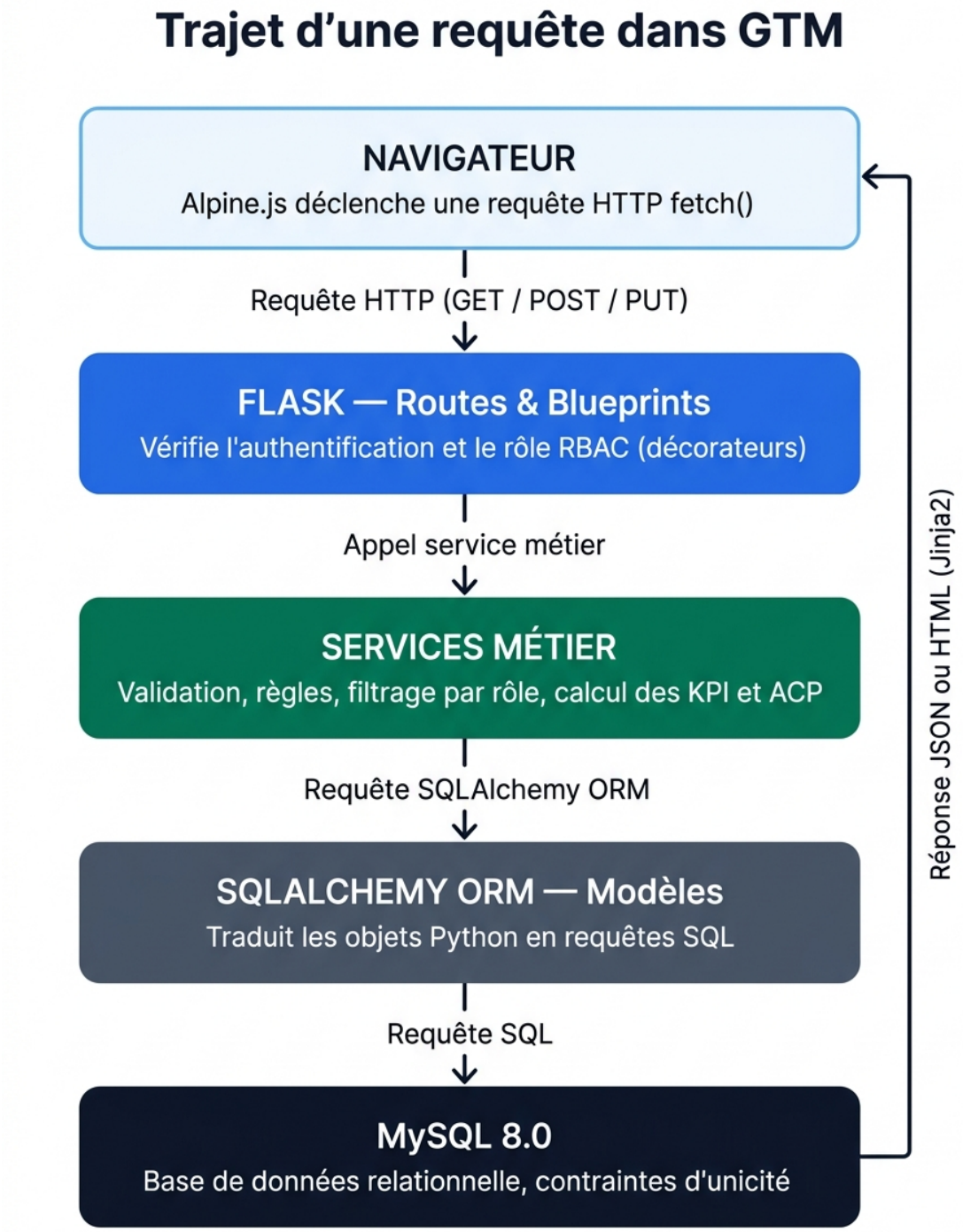


FIGURE 5.1 – Trajet d'une requête entre les différentes technologies utilisées

Composant	**Technologie**
Langage backend	Python
Framework web	Flask
Accès aux données (ORM)	SQLAlchemy (Flask-SQLAlchemy)
Base de données	MySQL
Authentification	Flask-Login
Migrations de schéma	Flask-Migrate (Alembic)
Gabarits HTML	Jinja2
Style de l'interface	Tailwind CSS (via CDN)
Interactivité front-end	Alpine.js
Graphiques	Chart.js
Calcul de l'ACP	NumPy, pandas
Hachage des mots de passe	Werkzeug (PBKDF2-SHA256)
Exports Excel	openpyxl
Exports PDF	ReportLab

TABLE 5.1 – Technologies utilisées dans GTM

Chapitre 6

Implémentation et fonctionnalités

6.1 Introduction

Ce chapitre décrit comment l'architecture conçue au chapitre 4 a été traduite en code, puis présente les principales fonctionnalités accessibles dans GTM : organisation du code, API REST, exports, tableau de bord, points d'attention, profil utilisateur et tests automatisés.

6.2 Organisation du code

L'organisation reprend l'architecture en couches présentée précédemment. Le tableau 6.1 présente les principaux dossiers de l'application.

Cette organisation limite le mélange des responsabilités : une règle de validation métier n'est pas réécrite dans chaque route qui l'utilise. Les services jouent un rôle central dans la réutilisation de la logique commune, tandis que les routes se concentrent sur la réception des requêtes et le formatage des réponses.

6.3 API REST

L'API de GTM expose les principales ressources à travers des requêtes HTTP standard : `GET` pour consulter, `POST` pour créer, `PUT` pour modifier, `DELETE` pour supprimer. Cette organisation sépare nettement les données métier de l'interface qui les affiche.

Les principales ressources disponibles sont notamment : `/api/formations`, `/api/sessions`, `/api/clients`, `/api/participants` (avec import par lot), `/api/formateurs`, `/api/inscriptions`, `/api/utilisateurs`, `/api/domaines` et `/api/stats`.

Le tableau 6.2 présente les codes HTTP utilisés par l'API.

Les listes acceptent plusieurs paramètres de filtre combinables (par exemple `/api/sessions?domaine`
Ces filtres sont traduits directement en clauses SQL par SQLAlchemy pour optimiser les performances.

Dossier	Rôle
`app/models`	Entités et relations de la base de données avec SQLAlchemy (9 modèles)
`app/services`	Logique métier réutilisable, validations, calculs et filtrage des données
`app/routes`	Points d'entrée de l'API REST retournant du JSON
`app/blueprints`	Routes utilisées pour servir les pages HTML de l'application
`app/templates`	Gabarits HTML Jinja2 utilisés pour construire les différentes pages (23 fichiers)
`app/static`	Feuilles de style CSS, images et scripts JavaScript Alpine.js
`database`	Scripts DDL SQL et données de démonstration déterministes
`scripts`	Scripts de maintenance, benchmarking et génération de données (6 fichiers)
`tests`	Suite de tests automatisés de l'application (22 fichiers)

TABLE 6.1 – Organisation des principaux dossiers de GTM

Code	Signification	Exemple dans GTM
200	Succès	Consultation réussie d'une liste ou d'une fiche
201	Ressource créée	Création réussie d'une nouvelle ressource
204	Succès sans contenu	Suppression réussie
400	Requête invalide	Donnée obligatoire absente ou format incorrect
401	Authentification requise	Utilisateur non connecté ou session expirée
403	Accès refusé	Rôle insuffisant ou tentative d'écriture d'un Formateur
404	Ressource introuvable	Identifiant qui ne correspond à aucune ressource
409	Conflit métier	Doublon d'inscription, session complète ou suppression avec dépendances
500	Erreur interne	Erreur inattendue côté serveur

TABLE 6.2 – Codes HTTP utilisés par l'API GTM et leur signification

6.4 Exports de données

Le format **CSV** fournit les données sous une forme brute, avec l'encodage `utf-8-sig` garantissant l'affichage correct des accents dans Microsoft Excel.

Les fichiers **Excel (XLSX)** sont générés avec `openpyxl`, avec une mise en forme professionnelle aux couleurs de Galaxy Solutions (en-têtes, alternance de couleurs et largeurs de colonnes ajustées).

ReportLab génère les documents **PDF** : les feuilles d'émargement officielles associées à chaque session et le rapport de pilotage décisionnel issu du tableau de bord.

6.5 Tableau de bord

Le tableau de bord rassemble les informations indispensables au suivi de l'activité de formation.

6.5.1 Indicateurs clés

Six indicateurs principaux sont calculés dynamiquement côté serveur : le nombre de sessions actives, le nombre de clients actifs sur les six derniers mois, le nombre de participants distincts formés, le nombre de formations dispensées, le taux moyen de remplissage et le nombre de formateurs mobilisés.

6.5.2 Filtres et graphiques

Le tableau de bord permet de croiser quatre critères de filtrage : l'année, le domaine, le client et le formateur.

Les indicateurs sont complétés par cinq graphiques interactifs animés (Chart.js) : répartition des inscriptions par domaine, activité par client, activité par formateur, évolution mensuelle des inscriptions et jauge de remplissage globale.

6.5.3 Espace personnalisé Formateur

Le rôle Formateur dispose d'une vue dédiée accessible sur `/dashboard-formateur`, alimentée par l'endpoint sécurisé `/api/stats/kpi-formateur`. Il y consulte ses quatre KPI personnels (sessions animées, sessions à venir, taux de remplissage moyen et participants formés) ainsi que la liste de ses prochaines sessions.

6.6 Points d'attention

Un service dédié (`points_attention_service.py`) analyse en temps réel l'activité pour dégager des alertes automatiques :

- **Sessions à risque (niveau danger) :** session dont la date de début est dans les 7 prochains jours avec un taux de remplissage strictement inférieur à 40 %.
- **Clients inactifs (niveau warning) :** entreprise cliente n'ayant enregistré aucune inscription confirmée sur les 6 derniers mois.
- **Tendance d'activité (niveau info) :** évolution du volume d'inscriptions supérieure à 5 % sur les trois derniers mois comparés aux trois mois précédents.

Ces alertes sont consultables dans le centre de notifications et via le badge dynamique de la barre supérieure.

6.7 Profil utilisateur

Chaque utilisateur connecté dispose d'un espace modal accessible depuis l'en-tête pour consulter son profil et modifier son mot de passe en saisissant son mot de passe actuel (figure 6.1).

6.8 Tests automatisés

La solution comprend une suite de **166 tests automatisés répartis dans 22 fichiers de tests** sous Pytest, exécutée sur une base de données de test séparée. Ces tests couvrent l'ensemble du périmètre : authentification, parcours d'onboarding, règles RBAC, CRUD des entités, inscriptions, filtres combinés, calculs d'indicateurs, exports et module ACP.

6.9 Synthèse de l'implémentation

Les fonctionnalités développées répondent à la totalité des exigences du cahier des charges en associant gestion opérationnelle, sécurité des accès et outils de décision. Leur validation détaillée est présentée au chapitre 8.

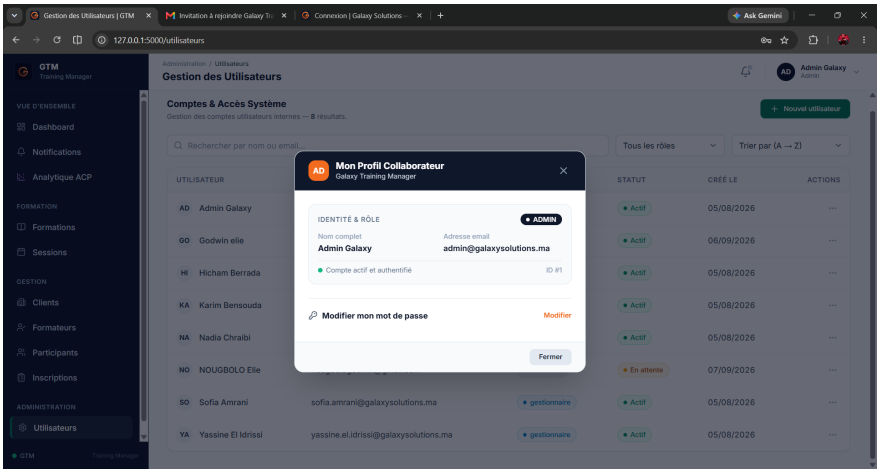


FIGURE 6.1 – Profil de l'utilisateur connecté et accès au changement de mot de passe

Chapitre 7

Analyse décisionnelle et ACP

7.1 Introduction

Le tableau de bord (chapitre 6) répond à des questions directes sur l'activité : nombre de sessions, nombre d'inscriptions, taux de remplissage. Mais lorsqu'un client suit plusieurs formations dans des proportions différentes, il devient difficile de comparer directement les profils à partir de simples tableaux.

L'**analyse en composantes principales (ACP)** a donc été intégrée à GTM pour explorer les ressemblances et les différences entre profils de clients, à partir de leurs historiques de formations [19, 20]. L'objectif n'est pas de prévoir le comportement futur des entreprises, mais de réduire la dimension des données pour faciliter leur interprétation visuelle.

7.2 Principe de l'ACP

Chaque client peut être décrit par son activité sur les différentes formations disponibles. Avec 12 formations dans le jeu de données de démonstration, un client peut être représenté par 12 variables : comparer directement tous les profils devient difficile quand le nombre de variables augmente.

L'ACP transforme ces variables en un nombre réduit d'axes, les **composantes principales**, qui conservent une partie importante de l'information des données originales [19]. Dans GTM, les deux premiers axes forment un plan factoriel $(F1, F2)$, qui permet de repérer des clients aux profils proches ou distincts, et la répartition générale des profils.

L'ACP est un outil d'**analyse exploratoire**, pas un modèle prédictif : elle ne permet pas, à elle seule, de prévoir les futures inscriptions d'un client.

7.3 Données et méthode de calcul

La matrice de données croise les clients et les formations : chaque ligne correspond à un client, chaque colonne à une formation, et x_{ij} représente le nombre d'inscriptions confirmées du client i à la formation j. Dans le jeu de données étudié, cette matrice contient **30 clients** et **12 formations**. Seules les inscriptions confirmées sont utilisées, pour rester cohérent avec le calcul du taux de remplissage (section 4.7.1).

Le calcul se déroule en quatre étapes.

1. Standardisation. Chaque colonne est standardisée par un score réduit (z-score) :

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j} \tag{7.1}$$

Cela évite qu'une formation avec naturellement plus d'inscriptions domine l'analyse uniquement à cause de son échelle.

2. Matrice de corrélation. À partir de la matrice standardisée Z, on construit $R = \frac{1}{n} Z^\top Z$, où n est le nombre de clients. R décrit les relations entre les formations : une corrélation positive élevée entre deux formations signifie que les clients actifs sur l'une le sont souvent aussi sur l'autre.

3. Valeurs et vecteurs propres. On décompose R en résolvant $Rv = \lambda v$. Les valeurs propres λ indiquent la variance expliquée par chaque axe ; les axes sont classés par ordre décroissant, $F1$ ayant la valeur propre la plus élevée et $F2$ la deuxième.

4. Projection des clients. Chaque client est ensuite projeté sur le plan $(F1, F2)$, avec des coordonnées correspondant à sa position sur les deux axes (figure 7.1).

Deux clients proches sur ce plan ont des combinaisons de formations relativement similaires selon ces deux axes. Il faut rester prudent : une proximité sur le plan ne signifie pas que les profils sont identiques dans l'espace complet, puisque les axes affichés ne représentent qu'une partie de l'information totale.

7.4 Résultats obtenus sur le jeu de données

Le calcul a été réalisé sur **30 clients**, **12 formations** et **484 inscriptions confirmées**. Les deux premiers axes expliquent :

— **F1 : 18,5 %** de l'inertie totale ;
— **F2 : 15,8 %** de l'inertie totale ;
— **F1 + F2 : 34,3 %** de l'inertie totale.

Cette part cumulée de 34,3 % montre que le plan $(F1, F2)$ ne représente pas la totalité des différences entre les profils. La visualisation reste utile pour une première lecture, mais il faut éviter d'interpréter les positions des clients comme une représentation complète de leur activité.

Selon les résultats calculés par l'application, **Marjane Holding** est le client présentant le profil le plus atypique parmi les clients suffisamment bien représentés sur le plan factoriel, avec une excellente qualité de représentation de $\cos^2 \approx 0{,}89$ (89 % de son information captée par les deux premiers axes). Son positionnement très marqué sur l'axe $F1$ ($F1 = -5{,}51$) s'explique par un volume global d'inscriptions particulièrement élevé sur plusieurs thématiques à forte contribution (Management Agile, Python et Sécurité).

7.5 Qualité de représentation et contributions

Le **cos²** mesure la qualité de représentation d'un client sur les axes sélectionnés : proche de 1, il indique que la position observée représente bien le profil initial ; faible, il indique que le point est mal représenté, et qu'il serait risqué de tirer une conclusion forte à partir de sa seule position sur le graphique.

La **contribution** indique dans quelle mesure un client ou une formation participe à la construction d'un axe. Dans GTM, les contributions sont calculées et affichées dans le résultat détaillé de l'ACP (figure 7.2).

L'orientation et la signification précise des axes dépendent des formations qui y contribuent le plus : il n'est pas correct d'attribuer à $F1$ ou $F2$ une signification métier arbitraire sans examiner ces contributions [19]. Dans GTM, les contributions affichées permettent d'identifier quelles formations structurent principalement chaque axe, mais cette lecture reste propre au jeu de données étudié et n'est pas généralisée automatiquement par l'application.

7.6 Synthèse automatique et seuils de rigueur

L'application propose une synthèse automatique soumise à des critères statistiques stricts : pour qu'une proximité entre deux clients soit validée et mise en avant, l'algorithme exige que chacun des deux clients possède une qualité de représentation minimale d'au moins $\cos^2 \geq 0{,}50$ (50 %). Dans le jeu de données analysé, seuls 3 clients dépassent ce seuil de fiabilité, et aucun couple ne vérifie simultanément cette condition de proximité fiable. Le système indique alors explicitement qu'aucune paire proche n'est validée, évitant ainsi de fausses interprétations visuelles sur des points mal projetés.

Le calcul gère également les cas particuliers : une formation avec une variance nulle ne peut pas être standardisée normalement (écart-type égal à zéro), et le nombre de clients disponibles peut être insuffisant pour une analyse pertinente. Ces situations font partie des cas limites couverts par les tests automatisés.

7.7 Limites de l'analyse

La première limite est la part d'information représentée : avec 34,3 % d'inertie cumulée pour $F1$ et $F2$, le plan affiché doit être utilisé comme un outil de lecture exploratoire, pas comme une représentation complète des profils [19, 20].

La deuxième limite est l'absence de filtrage par période ou domaine : contrairement à certains graphiques du tableau de bord, l'ACP est calculée sur l'ensemble des données disponibles, pour garder un nombre de clients suffisant. Un filtrage trop restrictif pourrait réduire le nombre d'individus disponibles et rendre l'analyse moins stable. Une évolution future pourrait proposer des filtres, sous réserve de vérifier que le nombre d'observations reste suffisant.

Enfin, l'interprétation des contributions reste principalement manuelle : une évolution future pourrait les exploiter pour générer automatiquement une explication plus précise des axes, à condition de garder une interprétation statistiquement correcte.

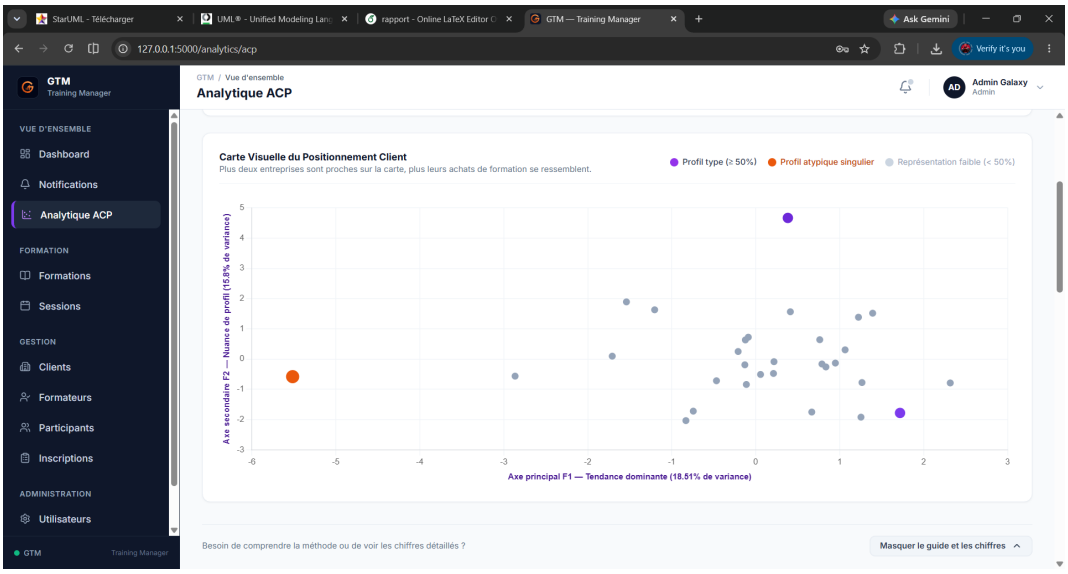


FIGURE 7.1 – Représentation des clients sur le plan factoriel $(F1, F2)$

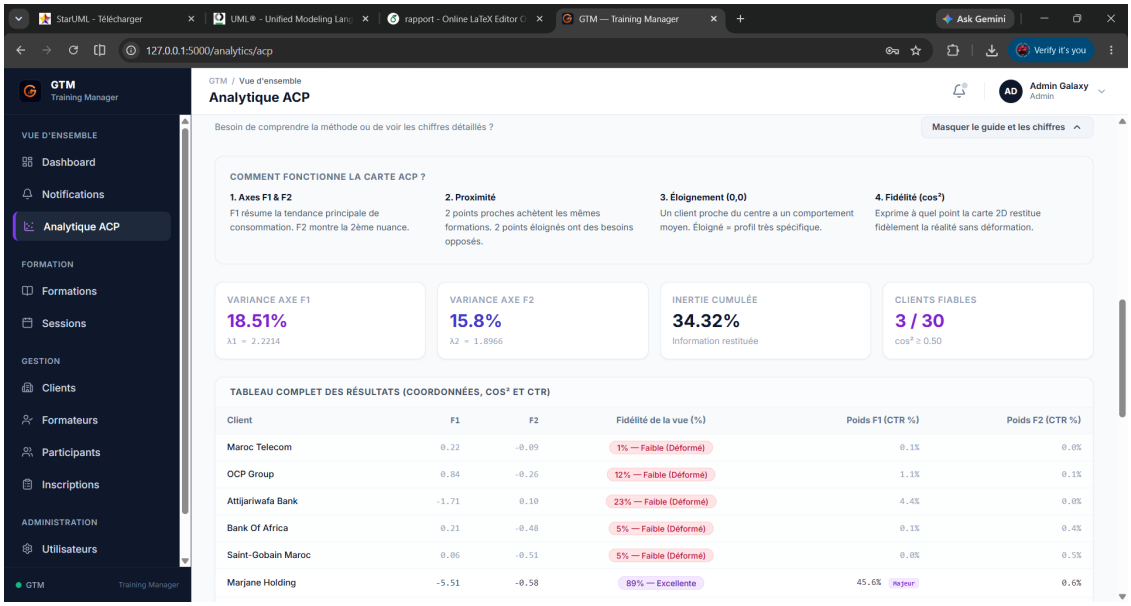


FIGURE 7.2 – Coordonnées, cos² et contributions calculés pour les clients

7.8 Bilan

L'ACP complète les indicateurs classiques du tableau de bord en proposant une autre manière de lire les données. Les deux premiers axes représentent 34,3 % de l'inertie totale : cette représentation reste partielle, mais elle permet d'explorer certaines proximités ou différences entre clients, à condition de tenir compte de la qualité de représentation et de la quantité d'information conservée. Dans GTM, l'ACP est donc principalement un outil d'exploration et d'aide à la lecture des profils, complémentaire aux indicateurs du tableau de bord.

Chapitre 8

Tests, audit et bilan critique

8.1 Introduction

Après l'implémentation des fonctionnalités, une phase d'évaluation a été menée selon deux démarches complémentaires : l'exécution d'une suite de tests automatisés couvrant les règles de gestion, et une relecture critique du code pour vérifier l'exacte conformité de l'application.

8.2 Démonstration fonctionnelle

Le tableau de bord (figure 8.1) restitue les six KPI, les graphiques d'activité et les points d'attention avec synchronisation dynamique des filtres.

La fiche d'une session (figure 8.2) présente les informations de planification, le taux de remplissage et les inscriptions, avec blocage automatique des inscriptions confirmées en cas de session complète.

Le compte Formateur (figure 8.3) dispose d'un périmètre strictement isolé : seules les sessions qu'il anime lui sont accessibles, et toute tentative d'accès non autorisé renvoie un code HTTP 403.

8.3 Résultats des tests automatisés

La suite de validation comprend **166 tests automatisés répartis dans 22 fichiers de tests** sous Pytest, exécutés sur une base de données de test dédiée.

Ces tests couvrent l'intégralité du cycle de l'application :

— **Authentification & Onboarding :** connexion, tokens d'activation 48h, hachage PBKDF2 et changement de mot de passe.

— **Contrôle d'accès RBAC :** permissions par rôle et interdiction formelle d'écriture pour le Formateur.

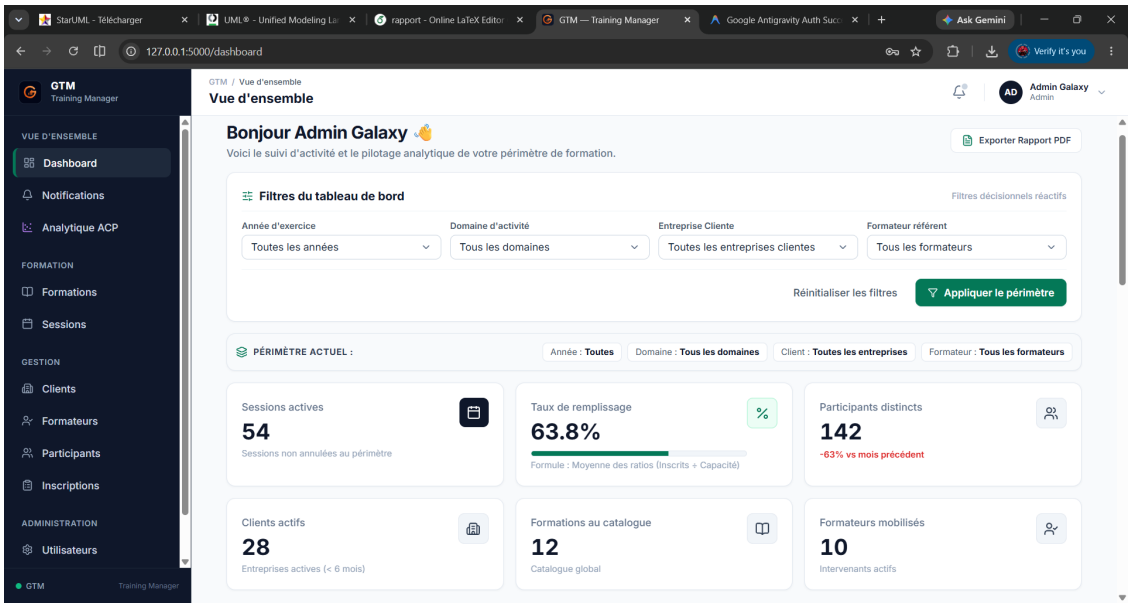


FIGURE 8.1 – Tableau de bord — vue Admin/Gestionnaire

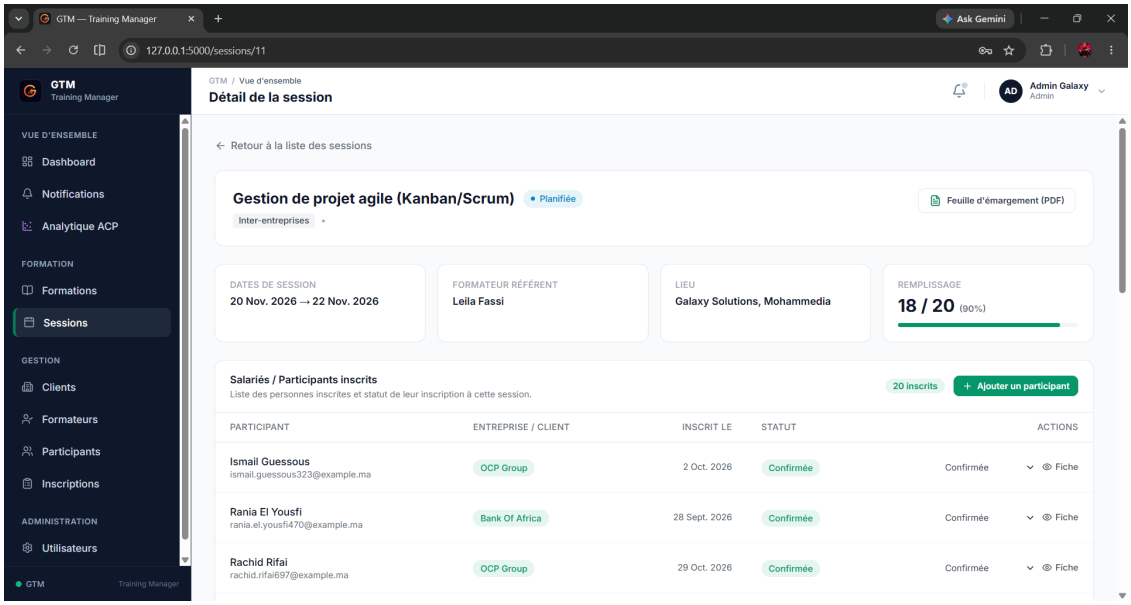


FIGURE 8.2 – Fiche détaillée d'une session avec taux de remplissage

— **Règles métier & Intégrité :** unicité des inscriptions, gestion des capacités, listes d'attente et suppressions sécurisées.
— **Imports, Exports & ACP :** validation des imports CSV/XLSX, génération des PDF ReportLab et exactitude des calculs factoriels.

L'ensemble des 166 tests s'exécute avec un taux de réussite de 100 %.

8.4 Bilan critique et audit interne

8.4.1 Démarche de vérification

À la fin du stage, j'ai mené une relecture critique approfondie du code pour comparer les règles attendues, le comportement réel des services et la documentation.

8.4.2 Points forts confirmés

La relecture a validé plusieurs points clés d'ingénierie : l'architecture en couches sépare nettement modèles, services et routes ; les contraintes SQL protègent l'intégrité de la base ; le parcours d'onboarding est sécurisé sans mot de passe temporaire en clair ; les contrôles de sécurité sont appliqués systématiquement côté backend ; et les erreurs renvoient des codes HTTP explicites.

8.4.3 Limite n°1 : calcul du statut d'activité client

La première anomalie identifiée concernait le calcul du statut d'activité sur la fiche d'un client. Alors que la liste des sessions était correctement filtrée pour le Formateur, la fonction calculant le **statut d'activité** et la **date de dernière activité** ne prenait pas en compte l'utilisateur connecté : elle évaluait donc l'activité globale de l'entreprise, y compris sur des sessions animées par d'autres formateurs.

Correction apportée : la fonction a été modifiée pour recevoir l'identifiant de l'utilisateur connecté et restreindre le calcul aux seules sessions visibles par ce Formateur. Un test unitaire spécifique a été ajouté pour valider ce comportement.

8.4.4 Limite n°2 : harmonisation documentaire

La deuxième anomalie concernait un décalage entre la documentation et les seuils réels du service de points d'attention (tableau 8.1).

Correction apportée : la documentation et les tests ont été parfaitement alignés avec le code opérationnel.

8.5 Autres limites et perspectives

La relecture a identifié des pistes d'amélioration pour une future mise en production : l'ajout d'un système de limitation des tentatives de connexion (anti-brute-force), la mise en place d'une pagination côté serveur sur les listes volumineuses, et la réalisation d'une démarche formelle de conformité à la loi 09-08 (CNDP) lors du passage sur des données réelles.

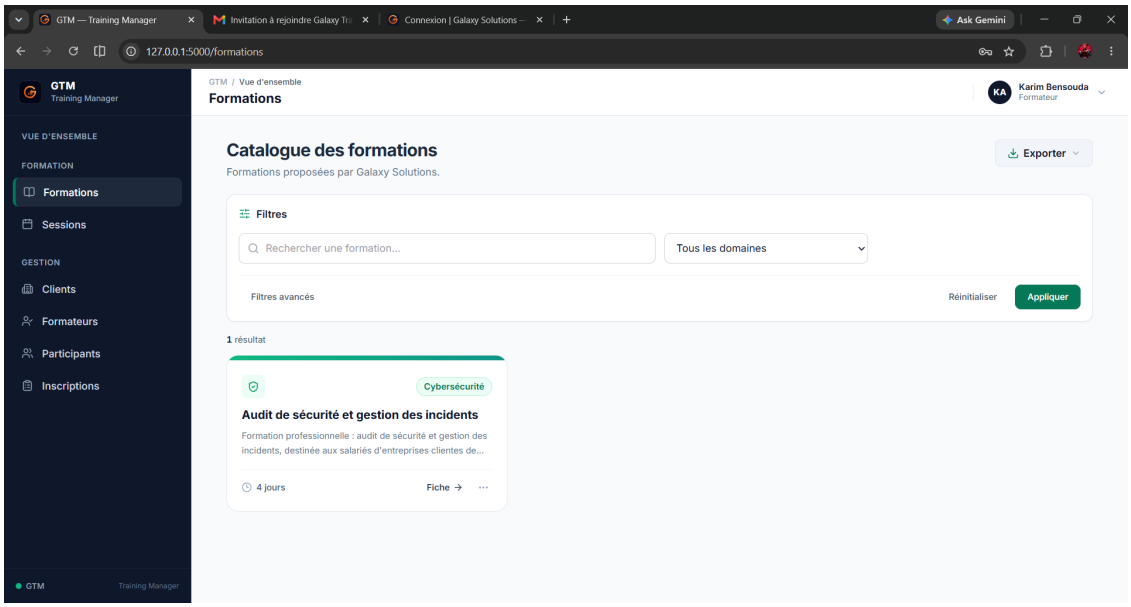


FIGURE 8.3 – Vue d'un compte Formateur, restreinte à son propre périmètre

Règle	**Documentation initiale**	**Comportement réel du code**
Session sous-remplie	Seuil de 15 jours, niveau *warning*	Seuil de 7 jours, niveau *danger*
Client inactif	Niveau *info*	Niveau *warning*
Session en surréservation	Mentionnée théoriquement	Non implémentée dans le service

TABLE 8.1 – Alignement des règles de points d'attention

8.6 Conclusion du bilan

Cette démarche de validation a confirmé que GTM est une application stable, fonctionnelle et conforme aux attentes. Elle m'a permis de prendre conscience qu'un développement logiciel rigoureux exige des vérifications continues entre les besoins, le code et les tests.

Conclusion et perspectives

Ce stage m'a permis de mener à bien un projet complet de développement logiciel, depuis l'expression des besoins jusqu'à la validation d'une application web professionnelle. La réalisation de GTM m'a conduit à aborder des problématiques variées : modélisation relationnelle, architecture en couches, sécurité applicative, interface réactive, analyse de données et tests automatisés.

Cette mission a représenté une expérience technique et méthodologique importante. Elle m'a appris à privilégier la solidité du socle métier, à valider chaque étape avant de construire la suivante, et à vérifier systématiquement la cohérence entre les règles de gestion et leur implémentation réelle.

Perspectives

Plusieurs pistes d'évolution pourraient enrichir la solution dans le cadre d'un déploiement à grande échelle :

- **Sur le plan fonctionnel :** automatiser l'envoi d'emails transactionnels pour la relance des clients inactifs et les alertes sur les sessions à risque.
- **Sur le plan technique :** intégrer une pagination côté serveur pour les volumes très importants, renforcer la protection CSRF et mettre en place une limitation de débit sur les tentatives de connexion.
- **Sur le plan de l'analyse :** étendre l'ACP en permettant un filtrage dynamique par période tout en contrôlant la taille de l'échantillon.

Bibliographie

[1] Moodle, "Moodle Workplace Multi-tenancy Documentation." https://docs.moodle.org/502/en/Multi-tenancy, 2026. Consulté en juillet 2026.

[2] TalentLMS, "TalentLMS Pricing and Enterprise Plans." https://www.talentlms.com/pricing, 2026. Consulté en juillet 2026.

[3] Docebo, "Docebo Enterprise Learning Platform Pricing." https://www.docebo.com/pricing/, 2026. Consulté en juillet 2026.

[4] OWASP Foundation, "OWASP Top 10 : The Ten Most Critical Web Application Security Risks." https://owasp.org/www-project-top-ten/, 2021. Version 2021, consultée en août 2026.

[5] OWASP Foundation, "OWASP API Security Top 10." https://owasp.org/API-Security/editions/2023/en/0x11-t10/, 2023. Consulté en août 2026.

[6] Python Software Foundation, "secrets — Generate Secure Random Numbers for Managing Secrets." https://docs.python.org/3/library/secrets.html, 2026. Documentation Python, consultée en août 2026.

[7] Python Software Foundation, "hmac — Keyed-Hashing for Message Authentication." https://docs.python.org/3/library/hmac.html, 2026. Documentation Python, consultée en août 2026.

[8] Pallets Projects, "Flask Documentation (v3.0)." https://flask.palletsprojects.com/, 2026. Documentation officielle, consultée en juillet 2026.

[9] SQLAlchemy Project, "SQLAlchemy 2.0 Documentation." https://docs.sqlalchemy.org/, 2026. Documentation officielle, consultée en juillet 2026.

[10] Oracle Corporation, "MySQL 8.4 Reference Manual." https://dev.mysql.com/doc/refman/8.4/en/, 2026. Documentation officielle, consultée en juillet 2026.

[11] Flask-Login Contributors, "Flask-Login Documentation." https://flask-login.readthedocs.io/, 2026. Documentation, consultée en août 2026.

[12] Flask-Migrate Contributors, "Flask-Migrate Documentation." https://flask-migrate.readthedocs.io/, 2026. Consulté en août 2026.

[13] SQLAlchemy Project, "Alembic Documentation." https://alembic.sqlalchemy.org/, 2026. Documentation officielle, consultée en août 2026.

[14] Pallets Projects, "Jinja Documentation." https://jinja.palletsprojects.com/, 2026. Documentation officielle, consultée en juillet 2026.

[15] Alpine.js, "Alpine.js Documentation." https://alpinejs.dev/, 2026. Documentation officielle, consultée en août 2026.

[16] Tailwind Labs, "Tailwind CSS Documentation." https://tailwindcss.com/docs, 2026. Documentation officielle, consultée en août 2026.

[17] Chart.js Contributors, "Chart.js Documentation." https://www.chartjs.org/docs/, 2026. Documentation officielle, consultée en août 2026.

[18] NumPy Developers, "NumPy Documentation." https://numpy.org/doc/, 2026. Documentation officielle, consultée en août 2026.

[19] I. T. Jolliffe and J. Cadima, "Principal component analysis : a review and recent developments," *Philosophical Transactions of the Royal Society A : Mathematical, Physical and Engineering Sciences*, vol. 374, no. 2065, p. 20150202, 2016.

[20] I. T. Jolliffe, *Principal Component Analysis*. New York : Springer, 2 ed., 2002.